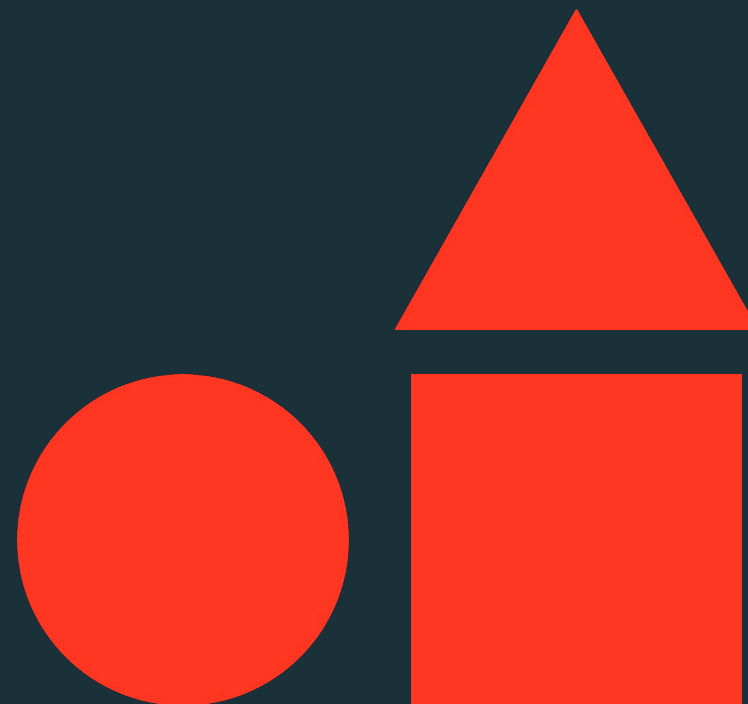


# Genie API Best Practices

Optimising reliability and performance for Genie API use cases

---



# Background

## Why implement Genie API Best Practices?

### ! Problem Context

- Customers are integrating Genie into **production-level AI applications** via the [Genie Conversation API](#)
- Free-tier requests are currently limited to **5 queries per minute**, per workspace (queries pertain only to POST requests to ask questions)
- Some customers have observed **throttling** (429 – TOO MANY REQUESTS errors), causing poor user experience
- Upgrading the QPM limit costs **\$\$\$**, which can be a barrier for some customers
- Genie Product team needs to see efforts to implement some best practices before upgrading QPM

### ✓ How best practices help the customer

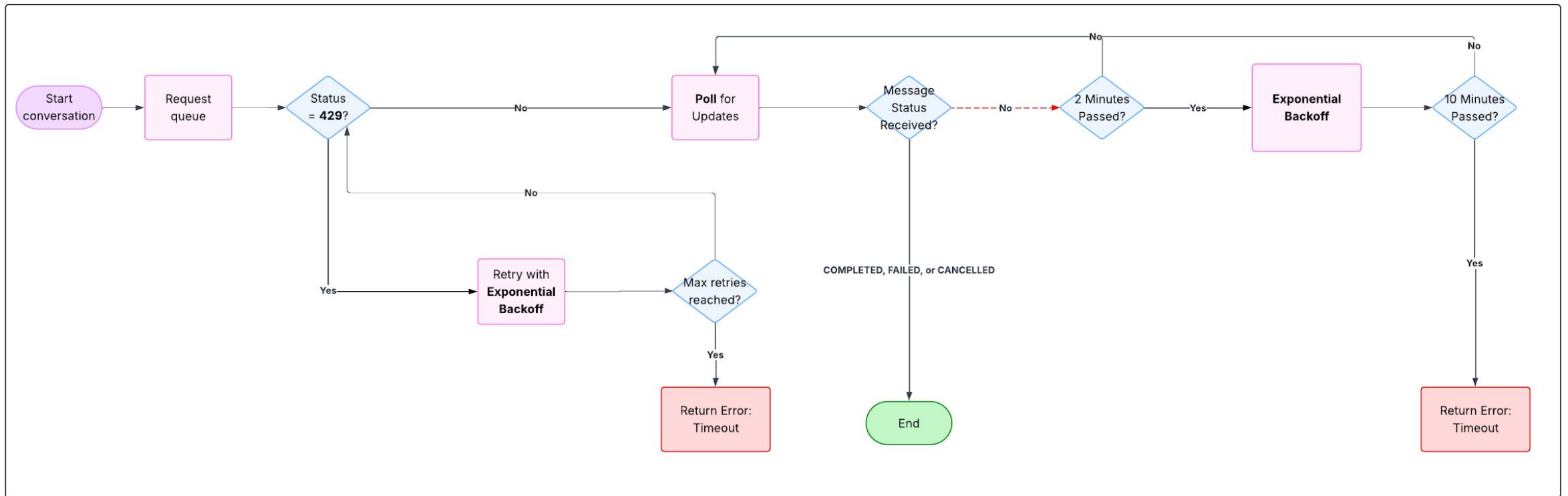
- **Better end user experience** – reduces risk of application outages or dropped conversations
- **Reduces IT overhead** – less time spent troubleshooting, debugging
- **Reduces TCO** for Genie – customer does not need to spend \$\$\$ for provisioned throughput
- **Provides scalability** for future use cases as more Genie spaces are created within the same workspace



# Best Practices

Process flow diagram based on [Databricks Genie API documentation](#)

## Genie API Best Practices with Polling and Exponential Backoff



Note: building a request queue can be an involved process; start with implementing polling and exponential backoff to enable quick wins

# Resources

## VERSION 0.1

- [Github](#)
- [Notebook](#) (do NOT run stress tests in Logfood!)
- [AWS Article](#) on how retry with backoff improves application stability
  - The article also goes in-depth into different exponential backoff + “jitter” implementations and how it improves request completion time

